

Exam Programming for Physics: Physics-Informed Neural Networks to Solve Differential Equations

Vasco Verwimp

28 May 2026

Contents

1	Introduction	2
1.1	Physics-Informed Neural Networks (PINNs)	2
1.1.1	Advantages And Disadvantages Of PINNs	3
1.1.2	Implementation Details	4
1.2	The Damped Harmonic Oscillator	4
1.2.1	Problem Description	4
1.2.2	Physical Characterization	5
1.2.3	Analytical Solution	5
1.3	Burgers' Equation	5
1.3.1	Problem Description	5
1.3.2	Physical Interpretation	6
1.3.3	Benchmark Initial Conditions	6
2	Understanding a Given Solution	7
3	Structure and Implementation	17
3.1	General Structure	17
3.1.1	Configuration and Inference	17
3.1.2	Data Generation and Sampling	18
3.1.3	Training and Saving	19
3.1.4	Evaluation and Plotting	20
3.2	Differences In Implementation Between The Damped Harmonic Oscillator And The Burgers' Equation	21
3.3	Hyperparameter Optimization	21
4	Results	22
4.1	Damped Harmonic Oscillator	22
4.2	Burgers' Equation	22
5	Conclusion	23

Chapter 1

Introduction

Traditional approaches to solving differential equations rely on analytical methods or classical numerical techniques such as finite difference methods, finite element methods, and spectral methods. However, these methods often struggle with high-dimensional problems, complex geometries, and inverse problems. In recent years, machine learning has emerged as a powerful alternative for solving differential equations, offering flexibility and scalability that traditional methods cannot provide.

Physics-Informed Neural Networks (PINNs) are a relatively new technique to solve problems described by ordinary or partial differential equations with boundary conditions. The idea gained traction in the late 1990s when several authors discussed solving these equations with a neural network by incorporating the equation directly into the loss function. During this era, the technique remained a niche academic curiosity because computers were not fast enough, and modern libraries (like PyTorch or TensorFlow) did not exist to handle the complex automatic differentiation required.

PINNs are applied to two classical differential equation problems:

1. **The Damped Harmonic Oscillator:** A second-order ordinary differential equation (ODE) representing mechanical vibrations with friction.
2. **Burgers' Equation:** A nonlinear partial differential equation (PDE) that combines advection and diffusion processes.

We investigate how PINNs can solve these equations by training neural networks to simultaneously satisfy the observed data and the underlying physics constraints. This will be used to extrapolate into regions for which no observational data was given. As a benchmark, we will use a standard machine learning model: a simple neural network trained on data loss only. Additionally, we will compare different PINN implementations to assess their predictive capabilities.

1.1 Physics-Informed Neural Networks (PINNs)

A Physics-Informed Neural Network is a neural network that is trained to solve differential equations by incorporating the equation's physics directly into the loss function. The network learns to approximate the solution $u(x, t)$ such that it satisfies both:

1. The observed data (data loss, $\mathcal{L}_{\text{data}}$)
2. The differential equation (physics loss, $\mathcal{L}_{\text{phys}}$)

3. Initial and boundary conditions (constraint loss, $\mathcal{L}_{\text{ic}}$)

The key innovation that has greatly increased the ease of use of PINNs is automatic differentiation to compute derivatives. This allows us to directly enforce the physics constraints encoded in differential equations.

Let $u(x, t)$ denote the solution to a differential equation. A neural network $u_\theta(x, t)$ is trained to approximate this solution, where θ represents the network's parameters. The total loss function is:

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{data}}(\theta) + \lambda_{\text{phys}}\mathcal{L}_{\text{phys}}(\theta) + \lambda_{\text{ic}}\mathcal{L}_{\text{ic}}(\theta) \quad (1.1)$$

where:

- $\mathcal{L}_{\text{data}}$ is the **data loss**: measures the discrepancy between network predictions and observed data

$$\mathcal{L}_{\text{data}} = \frac{1}{N_{\text{data}}} \sum_{i=1}^{N_{\text{data}}} (u_\theta(x_i, t_i) - u_i^{\text{obs}})^2 \quad (1.2)$$

- $\mathcal{L}_{\text{phys}}$ is the **physics loss**: enforces the differential equation at randomly chosen points, called collocation points, in a certain region.

$$\mathcal{L}_{\text{phys}} = \frac{1}{N_{\text{col}}} \sum_{j=1}^{N_{\text{col}}} \left(\mathcal{F} \left(u_\theta, \frac{\partial u_\theta}{\partial x}, \frac{\partial u_\theta}{\partial t}, \dots \right) \Big|_{x=x_j, t=t_j} \right)^2 \quad (1.3)$$

where $\mathcal{F}$ represents the differential equation operator: the solution u has $\mathcal{F}(u(x, t))=0$ for $\forall x, t$.

- $\mathcal{L}_{\text{ic}}$ is the **initial/boundary condition loss**: enforces initial and boundary conditions

$$\mathcal{L}_{\text{ic}} = \frac{1}{N_{\text{ic}}} \sum_{k=1}^{N_{\text{ic}}} (u_\theta(x_k, t_0) - u_0(x_k))^2 \quad (1.4)$$

- λ_{phys} and λ_{ic} are **loss weights** controlling the trade-off between fitting data and satisfying physics.

1.1.1 Advantages And Disadvantages Of PINNs

Advantages

- + **No mesh required**: Unlike finite difference or finite element methods, PINNs do not require discretization of the spatial and temporal domains; collocation points can be chosen at random, which limits the discretization effects.
- + **Inverse problems**: Can be used to identify unknown physical parameters from data.
- + **Extendability**: The neural network learns about the workings of the physical system; this allows it to make meaningful predictions in regions that it was not trained for.

Disadvantages

- **Longer compute time:** The added calculation of gradients (via automatic differentiation) significantly increases computational cost compared to standard neural networks.
- **Hyperparameter sensitivity:** Performance is very sensitive to the choice of loss weights (λ_{phys} , λ_{ic}), network architecture, and optimizer settings. Careful tuning is required and can be time-consuming.
- **Discontinuities and shocks:** PINNs struggle with discontinuous solutions and shock fronts because derivatives can become extreme in these cases.

1.1.2 Implementation Details

In practice, a PINN is typically implemented using the following structure:

1. **Network architecture:** A fully-connected feedforward neural network with several hidden layers and activation functions.
2. **Collocation points:** Sample interior collocation points to evaluate the physics residual and boundary/initial condition points separately.
3. **Automatic differentiation:** Using frameworks like PyTorch or TensorFlow to compute any order derivatives.
4. **Optimization:** Train the network using gradient-based optimizer to minimize the combined loss function.

1.2 The Damped Harmonic Oscillator

1.2.1 Problem Description

The damped harmonic oscillator is one of the most fundamental systems in classical mechanics, describing the motion of a mass attached to a spring with damping. It is governed by the following second-order ordinary differential equation:

$$m\ddot{y}(t) + c\dot{y}(t) + ky(t) = 0 \quad (1.5)$$

where:

- m is the mass $[kg]$
- c is the damping coefficient $[kg/s]$
- k is the spring stiffness $[N/m]$
- $y(t)$ is the displacement $[m]$
- $\dot{y}(t) = \frac{dy}{dt}$ is the velocity $[m/s]$
- $\ddot{y}(t) = \frac{d^2y}{dt^2}$ is the acceleration $[m/s^2]$

The system is typically subject to initial conditions:

$$y(0) = y_0, \quad \dot{y}(0) = v_0 \quad (1.6)$$

1.2.2 Physical Characterization

The behavior of the damped harmonic oscillator can be characterized using two non-dimensional quantities:

The **undamped natural frequency** is given by:

$$\omega_0 = \sqrt{\frac{k}{m}} \quad (1.7)$$

The **damping ratio** is given by:

$$\zeta = \frac{c}{2\sqrt{mk}} = \frac{c}{2m\omega_0} \quad (1.8)$$

The damping ratio determines the system's behavior:

- **Underdamped** ($\zeta < 1$): Oscillatory response with exponential decay
- **Critically damped** ($\zeta = 1$): Fastest non-oscillatory return to equilibrium
- **Overdamped** ($\zeta > 1$): Slow non-oscillatory return to equilibrium

1.2.3 Analytical Solution

Each situation can be solved analytically:

- Overdamped: $\Rightarrow y(t) = Ae^{-\omega_0(\zeta + \sqrt{\zeta^2 - 1})t} + Be^{-\omega_0(\zeta - \sqrt{\zeta^2 - 1})t}$
- Critically damped: $\Rightarrow y(t) = (A + Bt)e^{-\omega_0 t}$
- Underdamped: $\Rightarrow y(t) = e^{-\zeta\omega_0 t} (A \cos \omega_d t + B \sin \omega_d t)$

where $\omega_d = \omega_0 \sqrt{1 - \zeta^2}$ is the **damped natural frequency**, and constants A , B are determined by initial conditions.

1.3 Burgers' Equation

1.3.1 Problem Description

Burgers' equation is a fundamental nonlinear partial differential equation that arises in fluid dynamics, gas dynamics, and heat conduction. It combines nonlinear advection with linear diffusion:

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2} \quad (1.9)$$

where:

- $u(x, t)$ is the displacement variable $[m/s]$
- $\nu > 0$ is the viscosity coefficient $[m^2/s]$
- x is the spatial coordinate $[m]$
- t is the temporal coordinate $[s]$

The equation is supplemented with an **initial condition**: $u(x, 0) = u_0(x)$.

1.3.2 Physical Interpretation

The three terms in Burgers' equation have clear physical meanings:

- $\frac{\partial u}{\partial t}$: **Time derivative** — rate of change of the solution
- $u \frac{\partial u}{\partial x}$: **Advection** — nonlinear transport of the solution field (sharpening effects)
- $\nu \frac{\partial^2 u}{\partial x^2}$: **Diffusion** — smoothing due to viscosity (dissipation)

The balance between advection (nonlinear steepening) and diffusion (smoothing) creates complex dynamics:

- Small ν : Advection dominates, steep fronts form and can develop shocks
- Moderate ν : Interplay between smoothing and transport
- Large ν : Diffusion dominates, smoothing then dissipation

1.3.3 Benchmark Initial Conditions

Three benchmark initial conditions are commonly used for testing Burgers' equation:

Gaussian Initial Condition

$$u(x, 0) = \exp\left(-\frac{x^2}{2}\right) \quad (1.10)$$

This represents a smooth, localized disturbance that is transported and/or diffused over time.

Step Up Function

$$u(x, 0) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases} \quad (1.11)$$

This represents a sharp discontinuity that smooths out and/or shows rarefaction over time.

N-Wave

$$u(x, 0) = \begin{cases} -x & \text{if } |x| < 1 \\ 0 & \text{otherwise} \end{cases} \quad (1.12)$$

This represents a linear wave-like structure that produces two lobes that evolve towards each other, producing a shockwave for small viscosities.

Chapter 2

Understanding a Given Solution

As a first exercise, we were given a list of questions to answer thoroughly. These questions pertain a given PINN implementation for the damped harmonic oscillator. The questions provide a better understanding in the inner workings of the script and possible limitations and errors that are present. These are separated into questions about the methods and about the code.

Questions about the methods

Methods Question 1

Identify the total loss function in the code and write its expression down in your report. Identify the different contributions to the loss function.

The loss function is implemented in the training loop as follows:

```
y_pred = model(t_obs_t)
l_data = torch.mean((y_pred - y_obs_t) ** 2)

if use_physics:
    l_phys = loss_physics(model, t_col_t)
    l_ic   = loss_ic(model, t_ic_t)
    loss_total = l_data + LAMBDA_PHYS * l_phys + LAMBDA_IC * l_ic
else:
    l_phys = torch.zeros(1, device=DEVICE)
    l_ic   = torch.zeros(1, device=DEVICE)
    loss_total = l_data
```

`loss_total` is used as the total loss function. If the use of physics is disabled, this only includes the data loss (`l_data`), which is the mean-squared error between the model prediction and the given training observation points.

In case the use of physics is enabled, two terms are added: `LAMBDA_IC * l_ic`, giving the initial condition loss weighted with a factor λ_{ic} , and `LAMBDA_PHYS * l_phys`, giving the physics loss weighted with a factor λ_{phys} . This is consistent with the standard implementation mentioned in the introduction, the calculation of the losses are as follows:

```
def loss_physics(model: nn.Module, t: torch.Tensor) -> torch.Tensor:
```



```

"""
L_physics = mean( r(t_i)^2 ) over collocation points


$$r(t) = m y_{\text{hat}}''(t) + c y_{\text{hat}}'(t) + k y_{\text{hat}}(t)$$


Both derivatives are obtained via automatic differentiation through the
network. create_graph=True on the first call keeps the computation graph
alive so the second differentiation can flow back through it.

The [0] unpacks the single-element tuple returned by autograd.grad
(the API always returns a tuple, one entry per input tensor).
"""
y_hat = model(t)

# first derivative dy/dt
dy = torch.autograd.grad(
    y_hat, t,
    grad_outputs=torch.ones_like(y_hat),
    create_graph=True
)[0]

# second derivative d2y/dt2
d2y = torch.autograd.grad(
    dy, t,
    grad_outputs=torch.ones_like(dy),
    create_graph=True
)[0]

residual = M * d2y + C * dy + K * y_hat
return torch.mean(residual ** 2)

def loss_ic(model: nn.Module, t: torch.Tensor) -> torch.Tensor:
    """
    L_ic = ( y_hat(0) - Y0 )^2 + ( y_hat'(0) - DY0 )^2

    Both initial conditions are enforced explicitly.
    y_hat'(0) is obtained via autograd because initial velocity
    is never directly observed in the data.
    """
    y_hat_0 = model(t)

    dy_0 = torch.autograd.grad(
        y_hat_0, t,
        grad_outputs=torch.ones_like(y_hat_0),
        create_graph=True
    )[0]

    return torch.mean((y_hat_0 - Y0) ** 2 + (dy_0 - DY0) ** 2)

```

To find the physics loss, the first and second time derivatives are calculated using the autograd PyTorch function. These are used to assign a physics residual to every collocation point by applying the differential equation operator $\mathcal{F}$. The mean of the squares of these residuals is taken and returned as the physics loss.

The initial condition loss is calculated using the mean of the squares of the error of the predicted value of $y(0)$ and $\dot{y}(0)$ when compared to their given truth values. Once again, the autograd PyTorch function is used to find the first time derivatives (which is needed to find $\dot{y}(0)$).

Methods Question 2

Is the inclusion of the initial condition loss essential for this particular example? When would it be essential?

The inclusion of the initial condition loss is not essential in this particular example, as it is not needed to uniquely determine the solution (in the underdamped case): two observations are (in most cases) sufficient to constraint the solution to one possibility. It would be essential if we didn't have observational data, or if multiple different solutions could provide very similar observation e.g. critically damped. Additionally, in the reverse problem the initial behavior is of great importance to accurately recover the physical parameters present in the observational data.

Methods Question 3

The loss function used here is based on the mean squared errors (MSE) between predictions and data points or initial condition, and on the mean squared residual (MSR) of the PDE for the collocation points. Why are these specific functional forms used, and would there be alternative functions that could replace the MSE or MSR?

These specific functional forms provide positive errors or residuals that grow when the predictions deviate from the target solution. By using a mean-square, the derivative of the loss is proportional to the difference in the required value (0 for residuals, targets for error) which greatly simplifies the computation needed during backpropagation.

Methods Question 4

What is the type of neural network used in this example? How many input nodes, output nodes and hidden layers does it have? How many parameters are then being optimized? What type of activation function is used and why? What quantity or quantities are provided at the input, and what quantity or quantities do we extract from the output of the network? Are the dimensions of this Neural Network architecture adequate, too small or too large for this example? How would you determine this?

The architecture used is a fully connected feedforward neural network (sometimes also called a Multi-Layer Perceptron or MLP). It is composed of one input layer of one

neuron (time), 4 hidden layers with each 32 neurons, and one output layer of one neuron (displacement). Each neuron (except for the input neuron) has its own bias (1 for the output layer and 4 times 32 for the hidden layer) and each connection has a weight (2 times 32 for the connections between the input layer and first hidden layer and between the last hidden layer and the output layer plus 3 times 32 squared for the connections between the hidden layers). This amounts to 3265 total trainable parameters.

The input is expected to be a $[N,1]$ tensor of time, this produces an $[N,1]$ output tensor of displacement. By enabling the input to be a vector rather than a single number, it is possible to parallelize the full calculation by using matrix multiplication.

The activation function used is the hyperbolic tangent function, this function transforms any value into a -1 to 1 range while keeping the ordering as it is strictly increasing. Additionally, it smooths out large differences at large values. Most importantly though, it has an easily computable first and second derivative:

$$\frac{\partial \tanh(y)}{\partial x} = (1 - \tanh^2(y)) \frac{\partial y}{\partial x}$$

$$\frac{\partial^2 \tanh(y)}{\partial x^2} = (1 - \tanh^2(y)) \frac{\partial^2 y}{\partial x^2} + (-2 \tanh(y))(1 - \tanh^2(y)) \frac{\partial y}{\partial x}$$

These are all just polynomials in function of the original tanh and derivatives of the input, this greatly simplifies the computation in backpropagation. Additionally, they allow for the calculation of $\frac{dy}{dt}$ and higher order derivatives by the full backpropagation of the result in function of the input parameter. Other activation functions (including ReLU) have derivatives that are either zero on large parts of their domain or have discontinuities. These prevent this backpropagation which is needed to evaluate the initial condition and physics loss that are essential in a PINN.

The dimensions of this architecture are adequate, this can be seen by comparing the validation loss to the training loss. If the training loss keeps increasing while the validation loss saturates, the network is too large and is overfitting; on the other hand if both the validation loss and training loss do not decrease to a low enough level, the network cannot represent the system as it is too small and underfitting. When we do this test with validation points that are kept separate from the training set, we see that both the validation loss and training loss keep decreasing. The training loss is better than the validation loss (in the case that these points have the same origin), which is to be expected as this is the quantity (or one of the quantities) the network was actively trying to minimize.

Methods Question 5

How many input samples are provided to the neural network model during each of its training cycles or epochs? What kind of samples are provided? Does the network receive again for each epoch the same exact samples? Would there be a better way to provide samples to this network for training over the various epochs?

The neural network model receives 15 randomly chosen timestamps with their corresponding y value to which normal distributed noise has been added. These timestamps and corresponding y values are the same for every epoch. This can be improved in two ways. One could provide new samples each epoch to provide additional training data, however this is not possible in every application (for example if the dataset of real life observations is limited). Another way is to supply only a randomly chosen subset of the

15 randomly chosen timestamps each training cycle. This technique prevents the network on training on only a few samples, which is possible as only two observations can fully fix the solution of the damped harmonic oscillator. This technique is known as mini-batching.

Methods Question 6

The example uses the Adam algorithm as the so-called optimizer. What is the purpose of this optimizer? What is specific about the Adam algorithm, and what are its main hyper-parameters? Are there viable alternatives for the Adam optimizer?

An optimizer is responsible for updating the parameters in the neural network: in this case just the weights and biases. It does this by using the results of a backpropagation to view how changing the parameter infinitesimally would change the result. The Adam optimizer is a momentum based optimizer: once it starts going one way it will continue for a certain time by remembering previous results. This is done by adapting the step size of parameters by the first and second moments of the gradients. The influence of past moments decays using two parameters: β_1 and β_2 for respectively the first and second moment. Additionally, the Adam optimizer takes in a base learning rate which scales the step size. Finally, there is one last hyperparameter ϵ which is a numerical correction that prevents the numerator of the step size of going to zero. The Adam optimizer is popular as it requires very little hyperparameter optimization: the default values for the hyperparameters β_1 , β_2 , and ϵ are adequate for almost every problem, thus only requiring the optimization of the learning rate.

Alternatively, one could use standard stochastic gradient descent, which only uses the gradient of the loss, but this can easily get stuck in local minima. Because of this, momentum based optimizer like RMSProp and AdaGrad are more viable alternatives, but both have disadvantages: RMSProp requires more hyperparameter tuning and AdaGrad can have slow convergence. The Adam optimizer is a compromise between these two which is why it is the most popular optimizer.

Methods Question 7

The example also uses an additional optimizer: a learning rate scheduler. What does this scheduler do? Is it a necessary ingredient for training a neural network? What are its main hyper-parameters? Are there alternatives that can be usefully applied in this example?

The learning rate supplied to the Adam optimizer comes from StepLR. It begins with a starting learning rate, which is its most important hyperparameter, the learning rate is then decreased with a decay factor γ every N epochs. These are two other influential hyperparameters.

The use of a learning rate scheduler is not strictly necessary, but can help fine-tune the results: a large starting learning rate may get close to the global minimum, but will struggle to eventually fall into the middle of it. For this reason, decreasing the learning rate is favorable. This decrease can alternatively be done in a continuous manner by using a power function like $e^{-kn_{\text{epoch}}}$ or only when the validation loss stops improving (Reduce-on plateau). Other popular choices like cosine annealing and cyclical learning

rates implement oscillating learning rates, which can help escape local minima.

Methods Question 8

How many epochs is the network trained for? How would you determine whether you have trained for too little or too many epochs?

The network is trained for 8000 epochs. By looking at the graph of validation loss over the epochs, one can determine whether this was too much or too little: if the validation loss flattens out way before the end of the training, the training was too long. On the other hand, if the validation loss is still decreasing when the training process is stopped, the training was too short.

Methods Question 9

Would it make sense to have a set of validation points? How would these look like? How would you use them?

A set of validation points allows one to investigate if the network is overfitting and when the neural network has been trained enough. How these would look depends on the goal: if one would like to simply quantify how accurate the neural network can reproduce the solution, these can be created by randomly choosing points in the training domain and providing the true analytic value as their truth value. On the other hand, if one is attempting to replicate a real data analysis scenario in which the number of observations is limited and imperfect then these need to be chosen out of the given observations. This can be replicated by first creating a larger observation set by randomly choosing timestamps and adding Gaussian noise to the true value. From this we can do a stratified splitting of the observation set into validation and training points. This makes sure that the validation points are spread out over the full training interval (see later for specifics).

Questions about the code

Code Question 1

Why are these two lines of code good, or bad, for:

```
torch.manual_seed(42)
np.random.seed(42)
```

These make it so the random choices that are being made will always have the same seed in every run. This causes the program to produce a consistent output, even though it is randomly sampling points.

Code Question 2

What is the purpose of the `np.ndarray` statements in this function definition:

```
def analytic(t: np.ndarray) -> np.ndarray:
```

The first applies to the `t` input: it indicates that this input is expected to be a NumPy array, this helps the readability and error-tracing of the code since most IDEs indicate when this is not matching the given input. The second explains what type the output of the function will be, which has the same benefits as the first.

Code Question 3

The data points are created randomly and then sorted before being offered to the neural net, using this statement:

```
t_obs = np.sort(np.random.uniform(0.1, T_TRAIN, N_OBS))
```

Is this good or bad?

The creation of random timestamps is good: this makes sure that there are no grid artifacts and that the neural network gets points that are not evenly spaced. This is especially the case with the damped harmonic oscillator, as this even spacing could show up in (partial) resonance with the solution which could for example cause all displacements on the random timestamps to be 0.

The sorting step is neutral: the datapoints are all passed into the neural network in parallel so sorting them does not really do anything. It can help with the plotting, but has no effect on the training results.

Code Question 4

The collocation points are created uniformly on a fixed grid, using this statement:

```
t_col = np.linspace(0.0, T_EXTRAP, N_COL)
```

Is this good or bad?

This line of code shows that the collocation points are chosen equally spaced all the way to the end of the extrapolating area. Choosing the collocation points in this way is not the best idea: the even spacing could show up in resonance and in doing so only sample part of the behavior of the solution. Additionally, taking these collocation points in the extrapolating area means that the extrapolation is not a true blind test: the neural network has been trained in minimizing the physics loss in this area.

Code Question 5

What is this statement doing? Is it necessary?

```
optimiser.zero_grad()
```

This statement resets the gradients of the optimizer, this is important as when `loss_total.backward()` is called, the gradient found is added to the existing gradient instead of setting it as a new value. If we remove this, gradients from past epochs will keep affecting the current epoch in an undesired manner.

Code Question 6

What are these statements doing in the training cycle? Is their order important?

```
loss_total.backward()  
optimiser.step()  
scheduler.step()
```

`loss_total.backward()` does the backward propagation: this process looks at how infinitesimal changes in each trainable parameter will affect the total loss, effectively calculating the gradient in this parameter space. It then adds it to the selected optimizer. Following this, `optimiser.step()` updates the trainable parameters by tweaking them with regard to their gradient and the given learning rate. After this, `scheduler.step()` steps the learning rate to the next setting.

The order of these statements is essential: the second needs the first to function, and if the stepping of the scheduler is arranged in front of the optimizer then the first learning rate setting is never used.

Code Question 7

Why do the collocation time input tensor and the initial condition time input tensor require the gradient option, while the input data tensors do not require a gradient?

```
t_obs_t = to_tensor(t_obs)  
y_obs_t = to_tensor(y_obs)  
t_col_t = to_tensor(t_col, requires_grad=True)  
t_ic_t = to_tensor(t_ic, requires_grad=True)
```

The `requires_grad` option is only necessary if one needs to calculate the derivative with respect to the tensor. This is not necessary for the observation points, as these are only used as inputs to the model. The collocation and initial condition points do require the calculation of the derivative: the initial condition sets $y(0)$ and $\dot{y}(0)$, so the calculation of the predicted $\dot{y}(0)$ demands the use of this option; similarly, the physics loss requires the calculation of $\dot{y}$ and $\ddot{y}$ at the collocation points.

Code Question 8

When presenting arrays of time points to the input of my neural network, I need to convert them into a PyTorch tensor, using this helper function:

```
def to_tensor(arr, requires_grad=False):
    return torch.tensor(arr, dtype=torch.float32,
                        requires_grad=requires_grad).unsqueeze(1)
```

What does the `unsqueeze(1)` method of the `torch.tensor` class do, and why do I need to do this?

The `unsqueeze` method adds a new dimension of a single index, depending on the number this dimension is x (for 0), y (for 1), or z (for 2). The input to the neural network should be a $[N,1]$ tensor, as discussed in methods question 4, but the NumPy array is just a $[N]$ array, so the `unsqueeze` makes sure to turn it into the right tensor.

Code Question 9

When computing the first and second derivatives of $y(t)$ we use the `torch.autograd` method. Explain why the `grad_outputs` are set to ones and why we need to take the zero-th element `[0]` of this output in the statements below:

```
dy = torch.autograd.grad(
    y_hat, t,
    grad_outputs=torch.ones_like(y_hat),
    create_graph=True)[0]
```

The `grad_outputs` option signifies the tensor v that the gradient needs to be multiplied by to create a scalar, shown as follows:

$$S = \sum_j v^j y_j$$

This is necessary as we require a vector as output which is created by taking the gradient of a scalar quantity. When v is all ones, then this becomes simply:

$$S = \sum_j y_j$$

The gradient with respect to t_i then becomes:

$$\nabla_i S = \frac{\partial \sum_j y_j}{\partial t_i}$$

It is reasonable to assume that only y_i is dependent on t_i , which finally gives us the result that we were looking for:

$$\nabla_i S = \frac{\partial y_i}{\partial t_i}$$

The function can also accept multiple tensors of variables to calculate the autograd to, because of this it will return a tuple of all of the gradients in the order in which the variables were supplied. In this case, only one variable is supplied, so it returns a tuple with only one element; the `[0]` is used to pick the first and only element out of this tuple.

Using these questions as a starting point, the damped harmonic oscillator script will be improved and adapted into a more flexible and clear framework. Additionally, the reverse problem will be executed. This approach will then be translated for the Burgers' equation, which will add a new dimension of complexity.

Chapter 3

Structure and Implementation

All 4 version of PINN scripts have very similar structure; firstly, this structure is shown and motivated. This will give a clear picture of the implementation of the models, without needing to be specific about one or the other. Following this, a few important differences are noted between the implementation of the damped harmonic oscillator and the Burgers' equation. Finally, we will conclude this chapter with the process of hyperparameter tuning for the different cases. This will provide a solid base for the understanding of the process that produces the results shown in the next chapter.

3.1 General Structure

The process is split over 3 scripts: `model.py`, `train.py` and `plot.py`. The first contains the architecture, an inference predictor, and a configuration dataclass that will hold all parameters necessary for creating and training a model. The second will create an object of this configuration class by parsing its argument when called, using the settings provided it will generate the data and collocation points required to train the models. It will then train and save the model alongside the configuration needed to create it, while also providing useful metrics to evaluate the model quantitatively. The third can then be called in order to create plots showing the resulting solutions, compared both through the model's generations and in comparison to other models and the true result.

3.1.1 Configuration and Inference

Each model is governed by a dedicated configuration dataclass (`DamperConfig` for the damped oscillator and `BurgerConfig` for Burgers' equation), which acts as the single source of truth for every parameter. Physical constants, domain boundaries, data-generation settings, architectural choices, optimizer hyper-parameters, loss weights, early-stopping criteria, snapshot epochs, and output paths are all declared as typed fields with explicit default values. Because the dataclass is instantiated once at the start of `train.py` and then passed by reference into every downstream function, there is no risk of different parts of the pipeline operating under inconsistent assumptions. By parsing arguments in `train.py`, the fields can be set from the terminal before anything else runs, so the full configuration used for a given run is always recoverable from the serialized bundle which is saved in the training results file.

Derived physical quantities are exposed as read-only `@property` attributes computed directly from the primary fields, ensuring they stay consistent regardless of how the

object was constructed. For the oscillator these include the undamped natural frequency $\omega_0 = \sqrt{k/m}$, the damping ratio $\zeta = c/(2\sqrt{mk})$, and the damped natural frequency $\omega_d = \omega_0\sqrt{1-\zeta^2}$. For Burgers' equation the analogous derived quantity is the inviscid shockwave formation time. This time can be derived using the method of characteristics; this would give the following result:

$$t_{\text{shock}} = \min -\frac{1}{u'(x, 0)}$$

This differs between initial-condition scenarios: it equals unity for the N-wave, $e^{1/2}$ for the Gaussian, and infinity for the step function (where characteristics do not cross). Computing these from primary fields rather than storing them separately eliminates an entire class of consistency bugs when hyper-parameters are swept.

The **FCNet** class defines a fully-connected network built from a configurable number of hidden layers, each followed by a **Tanh** activation, with a linear readout. For the one-dimensional oscillator the network maps a scalar time t to a scalar displacement $\hat{y}$; for the two-dimensional Burgers' problem it maps a concatenated (x, t) pair to a scalar velocity $\hat{u}$. In the reverse implementations, the network additionally carries one or multiple trainable scalar parameters corresponding to physics based parameters that were used to generate the observational data. We have chosen to use the base-10 logarithm for this, as every parameter that we are estimating needs to be strictly positive, and in doing this the reverse transformation $10^{\hat{\theta}}$ is positive. This parameter is updated alongside the network weights during training. A **from_config** class method constructs the network directly from a configuration object, removing any need to pass architectural arguments separately.

The **Predictor** class provides a stateless CPU-side inference wrapper around **FCNet**. It accepts an optional checkpoint path at construction time and loads weights immediately if one is provided, this enables one to evaluate the model's evolution over the epochs during certain chosen checkpoints. Otherwise, the network is left at its random initialization and weights must be supplied later via **load_state_dict**. All forward passes are performed inside a **torch.no_grad()** context after moving inputs to **float32** tensors, and outputs are returned as plain NumPy arrays, which makes downstream plotting code free of device-management concerns. The class also includes static methods for computing RMSE and an approximate physics residual via NumPy gradient functionality. For the reverse problems, **predict_params** recovers the current estimate of the parameter by exponentiating the found estimate (which corresponds to the logarithm of the parameter).

3.1.2 Data Generation and Sampling

The **generate_data** function is responsible for producing every array and tensor that the training loop and plotting scripts will use. It is self-contained and returns a single dictionary holding both NumPy arrays and device-resident PyTorch tensors. Observations are drawn by sampling locations uniformly at random within the training window and evaluating the ground-truth solution at those locations, then corrupting the result with additive Gaussian noise of standard deviation σ .

In earlier versions, the observations would be split into a validation and training sets by stratified splitting. This is a technique that ensures the validation points are chosen over the whole training region, which is not necessarily always the case for random splitting. In the submitted version, validation points are used that are evenly spaced, without the

addition of noise. The number of validation points is also chosen to be greater than the number of training points. In doing this, the validation loss becomes more accurate: it directly shows how much the solution diverts from the truth value. On the other hand, in many real life scenarios it is not possible to arbitrarily generate new noiseless observations, as this requires the truth to be known. This is why the stratified splitting approach has remained present (although unused) in the final solution. This set is used only for early stopping; it is never plotted, because on a dense grid it would simply retrace the true-solution curve and add visual clutter without conveying additional information. The true values of the solution are provided by the `analytic` function when provided the necessary inputs: which configuration is used and the coordinates probed.

Two pools of collocation points are pre-allocated on the compute device at the start of training rather than being resampled every epoch. One pool covers the full domain including the extrapolation region, and the other is restricted to the training window. Both pools are large (of order 10^4 points) to ensure good coverage; at each epoch a random index vector selects n_{col} points from the appropriate pool, introducing stochasticity in the physics loss without the overhead of repeatedly calling a random number generator for the full pool. Initial-condition samples are likewise pre-allocated and these remain constant throughout training. Dense evaluation grids covering the full domain are constructed in NumPy and kept on the CPU; they are embedded in the returned dictionary for use by the plotting script but play no role during the training loop.

3.1.3 Training and Saving

The `train_model` function implements the complete training loop for a single model and is the only place in the codebase where gradient computation occurs. It accepts three Boolean flags: `use_ic`, `use_physics` and `extrapolated_physics`, that control which loss components are active and which collocation pool is sampled from, allowing the same function to train the data-only baseline, the physics-informed model with full-domain collocation, and the physics-informed model confined to the training window, without code duplication. At the start of the loop the model is moved to the target device, a fresh Adam optimizer is constructed with the specified learning-rate and momentum parameters, and a step learning-rate scheduler is attached that multiplies the rate by γ every fixed number of epochs. In the case of the reverse problem, the logarithm estimators are also assigned to the optimizer with their own separate learning rate `lr_param`.

The total loss at each epoch is the sum of three terms weighted by configurable coefficients (if both `use_physics` and `extrapolated_physics` are set to true):

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \lambda_{\text{phys}} \mathcal{L}_{\text{phys}} + \lambda_{\text{IC}} \mathcal{L}_{\text{IC}}.$$

The data loss $\mathcal{L}_{\text{data}}$ is the mean-squared error between the network's predictions at the observation locations and the noisy measurements. The physics loss $\mathcal{L}_{\text{phys}}$ is the mean-squared ODE or PDE residual evaluated at the collocation points using automatic differentiation: `create_graph=True` is passed to the first `autograd.grad` call so that the computation graph is kept alive through the second derivative, which is required for the damped-oscillator loss ($m\hat{y}'' + c\hat{y}' + k\hat{y}$) and the Burgers' diffusion term ($-\hat{\nu} \partial_{xx} \hat{u}$). The IC loss $\mathcal{L}_{\text{IC}}$ penalizes deviation of the network's output from the prescribed initial condition.

Checkpointing and early stopping are interleaved with the training loop at intervals of `log_every` epochs. At each logging step the model is switched to evaluation mode, the validation loss is computed inside a `no_grad` context, and all scalar losses are appended

to a history dictionary. If the validation loss has decreased by more than `min_delta` relative to the best value seen so far, the current weights are serialized to the checkpoint file together with the epoch index, the validation loss, and the configuration object; a patience counter is reset to zero. Otherwise, the counter is incremented and if it reaches the `patience` threshold the loop exits early. This scheme guarantees that the best weights are always on disk even if the run is interrupted, and that training terminates automatically when generalization ceases to improve, without requiring the user to specify the number of epochs precisely.

At epochs listed as `snapshot_epochs` in the configuration class, the model's state dictionary is copied to the CPU with a deep copy and stored in a `snapshots` dictionary keyed by epoch number. The CPU copy is taken by iterating over `model.state_dict().items()` and calling `.cpu()` on each tensor before the deep copy, so the live model on the compute device is never disturbed.

The calling code in `train_and_save_three` (or `train_and_save_both` for the reverse problems) trains all three relevant networks (standard machine learning neural network (not applicable with the reverse problem), PINN with physics extending into the extrapolated area, and a truly blind PINN). After this, it loads the best-val checkpoint into a `Predictor`, evaluates it on the dense plotting grid, computes RMSE and physics-residual metrics separately for the training and extrapolation intervals, and serializes all acquired metrics and data into a single `training_results.pt` bundle that `plot.py` can load without any knowledge of how training was run. Importantly, this also includes the configuration class so that the architecture of the networks trained are saved directly besides the actual network weights.

3.1.4 Evaluation and Plotting

`evaluate_model` will print the acquired metrics of the trained models by looking at the training results file that matches the given parameters (ω_0 and ζ for damped oscillator, `situation` ("Step", "Gaussian", or "N-wave") and viscosity for Burgers'). Additionally, it returns these metrics for the normal PINNs or the estimated parameters for the reverse problem PINNs.

The plotting script begins by loading the `training_results.pt` bundle with `torch.load` mapped to CPU, extracting the configuration object, all history dictionaries, snapshot dictionaries, pre-computed dense predictions, and scalar metrics. Fresh `Predictor` instances are constructed from the recovered configuration. This design means the plotting script can be run on any machine regardless of the device used during training, and it never needs to reconstruct the training data or re-run any forward passes beyond those required for the snapshot panels.

Multiple figures are made to show the results and the evolution of the network over the epochs:

- **Loss-figure:** the evolution of all loss components on a logarithmic scale, using color to encode the loss category and line style to distinguish between models.
- **Evolution over the epochs:** Each model is shown evolving by using the multiple snapshots taken during training.
- **Focus on extrapolation window:** In both cases, the extrapolation window is specifically shown, which helps in assessing the behavior of the solutions in this area of interest.

The plotting script parses the arguments to acquire the location of the training results file: one can provide the location directly via `--results`. Alternatively, if this argument is not set, one can supply in which folder to find it and which parameters are used via `--input_dir`, and for example for the Burgers' equation `--situation` and `--viscosity`. If any of these are missing, the default value is used.

3.2 Differences In Implementation Between The Damped Harmonic Oscillator And The Burgers' Equation

There are some important things to note that do differ between the two different cases. This includes both problems specific for each case and some presentation differences.

Firstly, in the case of the Burgers' problem the ground truth cannot be written in closed form for arbitrary initial conditions, so a finite-volume solver (`BurgersSolver`) is used to pre-compute a dense set of solution snapshots on a fine spatio-temporal grid extending well beyond the plotting domain. The solver result is cached to a folder as a compressed NumPy archive on the first run and reloaded on subsequent runs, avoiding repeated expensive computation during hyper-parameter sweeps. This gives the result on a discretized mesh, when a value is probed in the gaps of this mesh an interpolation is used to acquire the value. The solving method used is fully implicit Crank-Nicolson method to discretize the PDE, this then is applied using the Newton method. This class was constructed using the Claude generative AI, after creation the inference function that applies the interpolation and the time stepping were modified to correct the creation and interpolation of the computed grid. For boundary values we have decided to use Dirichlet boundary values, but at distant points outside the spatial bounds of the system so that these boundary conditions are not able to influence the behavior of the solution in the observed bounds. These analytical solutions were compared to literature results and showed a good match.

The damped oscillator

3.3 Hyperparameter Optimization

Chapter 4

Results

4.1 Damped Harmonic Oscillator

4.2 Burgers' Equation

Chapter 5

Conclusion